

PRÄDIKATE

Prädikate sind Aussagen über mathematische Objekte, die wahr oder falsch sein können. «Funktionen» mit booleschen Rückgabewerten: $P, Q(n), R(x, y, z)$.

$1 \Rightarrow B \Leftrightarrow (\neg B \Rightarrow A) \Leftrightarrow (A \wedge B) \vee (\neg A \wedge \neg B) \Leftrightarrow A \vee B$

$1 \Rightarrow B \Leftrightarrow (\neg A \wedge B) \Leftrightarrow A \wedge \neg B \Leftrightarrow (A \Rightarrow B) \wedge (B \Rightarrow C)$

Begriff

Bedeutung

Abtrennungsregel

$(A \wedge (A \Rightarrow B)) \Rightarrow B$

Begriff

Bedeutung

Kommutativität

$(A \wedge B) \Leftrightarrow (B \wedge A)$
 $(A \vee B) \Leftrightarrow (B \vee A)$

Begriff

Bedeutung

Assoziativität

$A \wedge (B \wedge C) \Leftrightarrow (A \wedge B) \wedge C$
 $A \vee (B \vee C) \Leftrightarrow (A \vee B) \vee C$

Begriff

Bedeutung

Distributivität

$A \wedge (B \vee C) \Leftrightarrow (A \wedge B) \vee (A \wedge C)$
 $A \vee (B \wedge C) \Leftrightarrow (A \vee B) \wedge (A \vee C)$

Begriff

Bedeutung

Absorption

$A \vee (A \wedge B) \Leftrightarrow A$
 $A \wedge (A \vee B) \Leftrightarrow A$

Begriff

Bedeutung

Idempotenz

$A \vee A = A$
 $A \wedge A = A$

Begriff

Bedeutung

Doppelte Negation

$\neg(\neg A) \Leftrightarrow \neg\neg A \Leftrightarrow A$

Begriff

Bedeutung

Konstanten

$W = \text{wahr}$
 $F = \text{falsch}$

Begriff

Bedeutung

de Morgan

$\neg(A \wedge B) \Leftrightarrow \neg A \vee \neg B$
 $\neg(A \vee B) \Leftrightarrow \neg A \wedge \neg B$

NORMALFORMEN

Normalformen (allgemein, **kanonische Formen**) helfen, das Vergleichsproblem zu lösen (ob zwei Aussagen dieselben sind).

$$P_1 \wedge \dots \wedge P_n = \forall i \in \{1, \dots, n\} (P_i) = \bigwedge_{i=1}^n P_i$$
$$P_1 \vee \dots \vee P_n = \exists i \in \{1, \dots, n\} (P_i) = \bigvee_{i=1}^n P_i$$

NEGATION

Nicht für alle = Es gibt einen Fall, für den nicht

$\neg \forall i \in \{1, \dots, n\} (P_i) \Leftrightarrow \neg (P_1 \wedge \dots \wedge P_n) \Leftrightarrow \neg P_1 \vee \dots \vee \neg P_n \Leftrightarrow \exists i \in \{1, \dots, n\} \neg P_i$

MENGEN

KONSTRUKTION

Grundmenge G , Prädikat $P(x)$. Konstruierte Menge $A = \{x \in G \mid P(x)\}$

OPERATIONEN

Begriff

Bedeutung

Vereinigung

$A \cup B = \{x \mid x \in A \vee x \in B\}$

Begriff

Bedeutung

Schnittmenge

$A \cap B = \{x \mid x \in A \wedge x \in B\}$

Begriff

Bedeutung

Komplement

$\bar{A} = \{x \mid x \notin A\}$

Begriff

Bedeutung

Differenz

$A \setminus B = \{x \in A \mid x \notin B\}$

TUPEL

$A \times B = \{(a, b) \mid a \in A \wedge b \in B\}$

n -Tupel:
$$\prod_{i=1}^n A_i = \{(a_1, a_2, \dots, a_n) \mid a_i \in A_i\}$$

BEWEISE

Eine Folge von logischen Schlüssen, die zeigen, dass die Aussage aus den gegebenen Voraussetzungen folgt.

Beweistechnik

Anwendungsfall

Konstruktiver Beweis

Liefert explizite «Lösung» / Algorithmus

Beweistechnik

Anwendungsfall

Widerspruchsbeweis

Geeignet für Unmöglichkeitss Aussagen. Z.B. $\sqrt{2} \notin \mathbb{Q}$

Beweistechnik

Anwendungsfall

Vollständige Induktion

Für Aussagen, die für alle $n \in \mathbb{N}$ gelten.

SPRACHEN

$A = (Q, \Sigma, \delta, q_0, F)$

Regulär

DEA
NEA
Regex

Kontextfrei

CFG
PDA
Palindrome
 $\{0^n 1^n \mid n \geq 0\}$

Turing-erkennbar

Primzahlen
 $\{a^n b^n c^n \mid n \geq 0\}$

Alle Sprachen

Überabzählbar unendlich
 $F \subset Q$

Zustände:

$Q = \{q_0, q_1, \dots, q_n\}$

Übergangsfunktion:

$\delta: Q \times \Sigma \rightarrow Q$

Startzustand:

$q_0 \in Q$

Akzeptierzustände:

$F \subset Q$

Wichtig: Ein Pfeil für jedes Zeichen in jedem Zustand für validen DEA

WÖRTER EINER REGULÄREN SPRACHE

ÜBERGANGSFUNKTIONEN FÜR WÖRTER

$\delta: Q \times \Sigma^* \rightarrow Q: (q, w = a_1, \dots, a_n) \mapsto \delta(\dots \delta(q, a_1), a_2), \dots, a_n)$

Übergänge ausgehend von q für Zeichen $a_1, \dots, a_n$ nacheinander anwenden.

WORT AKZEPTIEREN

Der DEA $A = (\Sigma, Q, q_0, \delta, F)$ **akzeptiert** das Wort $w \in \Sigma^*$, wenn er A von Startzustand in einen Akzeptierzustand $\delta(q_0, w) \in F$ überführt.

AKZEPTIERTE SPRACHE, REGULÄRE SPRACHE

Gegeben ein DEA $A = (\Sigma, Q, q_0, \delta, F)$. Die von A **akzeptierte Sprache** ist

$L(A) = \{w \in \Sigma^* \mid A \text{ akzeptiert } w\} = \{w \in \Sigma^* \mid \delta(q_0, w) \in F\}$

Die Sprache $L \subset \Sigma^*$ heisst **regulär**, wenn es einen DEA A gibt mit $L(A) = L$

Beispiel 3: DEA, der die Sprache $L = \{w \in \Sigma^* \mid w \text{ ist eine ungerade Zahl im Dreiersystem}\}$ akzeptiert

0,2

1

0,2

Start

g

u

0

1

2

g

g

u

g

u

g

u

MYHILL-NERODE AUTOMAT

Für ein Wort $w \in \Sigma^*$ setze $L(w) = \{w' \in \Sigma^* \mid ww' \in L\}$. Insbesondere $L(\varepsilon) = L$

Gegeben: reguläre Sprache L über Σ . Rekonstruiere A mit:

- $Q = \{L(w) \mid w \in \Sigma^*\}$
- $q_0 = L(\sigma) = L$
- $F = \{L(w) \in Q \mid \varepsilon \in L(w)\}$
- $\delta(L(w), a) = L(wa)$

Gleicher Zustand $\Leftrightarrow$ gleiches $L(w)$

Beispiel 4: $\Sigma = \{0, 1\}, L = \{w \in \Sigma^* \mid |w|_0 \text{ gerade}\}$

1

w

1

Start

q_0

$L(w)$

$L(\varepsilon) = L$

$L(0) = \{w \in \Sigma^* \mid |w|_0 \text{ ungerade}\}$

$L(1) = \{w \in \Sigma^* \mid |w|_0 \text{ gerade}\}$

Beispiel 1: Wortlänge

$|\varepsilon| = 0$

$|01010|_1 = 2$

$|a^3 b^5| = 8$

$|01010|_0 = 3$

$|01010|_3 = 0$

$|(1291)^7| = 7 \cdot |1291| = 7 \cdot 4 = 28$

Beispiel 2: Sprache

$L = \Sigma^* \subset \Sigma^*$

$L = \emptyset \subset \Sigma^*$, die leere Sprache

$\Sigma = \{0, 1\}$, Sprache aller Binärstrings: $L = \Sigma^*$

$\Sigma = \text{Unicode}, J = \{w \in \Sigma^* \mid w \text{ wird vom Java-Compiler akzeptiert}\}$

$M = \text{eine "Maschine"}, L(M) = \{w \in \Sigma^* \mid w \text{ wird von } M \text{ akzeptiert}\}$

Beobachtungen 1

1.1. $|w^n| = n \cdot |w|$

1.2. Zu jeder Maschine M gibt es eine Sprache $L(M)$

DETERMINISTISCHE ENDLICHE AUTOMATEN (DEA)

Definition

Tabellenform

Zustandsdiagramm von A

Beispiel 5

Start

q_0

0

q_0

q_1

q_2

q_3

q_1

q_2

q_3

q_2

q_3

Algorithmus

1) $q \in F, q \notin F$

2) Unterscheidbar nach Übergang

3) Iteration bis keine Änderung mehr

$\Rightarrow q_1 \text{ und } q_2 \text{ sind nicht unterscheidbar}$

PUMPING LEMMA

Ist L eine reguläre Sprache, dann gibt es $N \in \mathbb{N}$, die pumping length so, dass jedes Wort $w \in L$ mit $|w| \geq N$ in drei Teile $w = xyz$ zerlegt werden kann mit:

- 1) $|xy| \leq N$
- 2) $|y| > 0$
- 3) $xy^k z \in L \forall k \in \mathbb{N}$

Oder: genügend lange Wörter ($|w| \geq N$) einer regulären Sprache ($w \in L$) können alle in einem Anfangsstück der Länge N ($|xy| \leq N$) aufgepumpt werden ($xy^k z \in L$).

Was zeichnet reguläre Sprachen aus?

- Reguläre Ausdrücke
- Lange Wörter: * kommt im regulären Ausdruck vor
- Blöcke mit * können beliebig oft wiederholt werden
- $\Rightarrow$ Wörter können «aufgepumpt» werden

BEWEIS

Behauptung: Die Sprache $L \subset \Sigma^*$ ist nicht regulär.

Beweis (Widerspruch):

- 1) Annahme: L ist regulär
- 2) Gemäss Pumping Lemma gibt es die Pumping Length N
 - Es darf keine Annahme über die konkrete Grösse von N gemacht werden!
- 3) Wähle ein Wort $w \in L$ mit $|w| \geq N$
 - Die Definition **muss** N verwenden!
- 4) Aufteilung des Wortes gemäss Pumping Lemma
 - $w = xyz, |xy| \leq N, |y| > 0$
- 5) Auswirkung des Pumpens
 - $xy^k z \notin L$ für mindestens ein $k \in \mathbb{N}$ (mit Begründung!)
- 6) Widerspruch und Schlussfolgerung, dass die Annahme nicht zutreffen kann

Beispiel 6: $L = \{0^n 1^n \mid n \geq 0\}$ ist nicht regulär

1) Annahme: L ist regulär

2) $\exists N \in \mathbb{N}$, Pumping Length

3) $w = 0^N 1^N$

4) Unterteilung $w = xyz$

Beispiel 7: $L = \{w \in \Sigma^* \mid w \text{ endet mit zwei 0}\}$

DEA-Lösung

NEA-Lösung

Start

q_0

1

q_1

q_0

q_2

UMGANG MIT MEHRDEUTIGEN ÜBERGÄNGEN

a-Übergang von q_1 aus nicht eindeutig

Welchen Übergang soll man nehmen?

- Beide Möglichkeiten probieren und akzeptieren, falls eine der Möglichkeiten auf einen Akzeptierzustand führt.
- Zufallsgenerator (probabilistischer endlicher autom., PEA)

AutoSpr | FS26

Seite 1

4) Startvariable: S

Es ist nicht garantiert, dass es für die Ableitung eines Wortes nur einen einzigen Syntaxbaum gibt. Man sagt, die Grammatik ist **nicht eindeutig**, wenn sie mehrere Syntaxbäume für die gleichen Wörter erlaubt.

REGULÄRE OPERATIONEN

Die Klasse der kontextfreien Sprachen ist abgeschlossen unter regulären Operationen.

Reguläre Sprachen sind kontextfrei.

Grammatik für reguläre Operationen

Seien L_1 und L_2 kontextfreie Sprachen mit Grammatiken $G_i = (V_i, \Sigma, R_i, S_i)$. Wir nehmen an, dass die Variablen- und Regelmengen disjunkt sind, also $V_1 \cap V_2 = \emptyset \wedge R_1 \cap R_2 = \emptyset$. Zudem sei S_0 eine Variable, die nicht in $V_1 \cup V_2$ enthalten ist. Dann gilt:

Alternative $L_1 \mid L_2$ ist kontextfrei mit der Grammatik

$$G = (V_1 \cup V_2 \cup \{S_0\}, \Sigma, R = R_1 \cup R_2 \cup \{S_0 \rightarrow S_1 \mid S_2\}, S_0)$$

Verkettung $L_1 L_2$ ist kontextfrei mit der Grammatik

$$G = (V_1 \cup V_2 \cup \{S_0\}, \Sigma, R = R_1 \cup R_2 \cup \{S_0 \rightarrow S_1 S_2\}, S_0)$$

Operation L_1^ ist kontextfrei mit der Grammatik

$$G = (V_1 \cup \{S_0\}, \Sigma, R = R_1 \cup \{S_0 \rightarrow S_0 S_1, S_0 \rightarrow \varepsilon\}, S_0)$$

KONTEXTFREI

Regeln ohne Kontext

Es spielt keine Rolle, in welchem Kontext das Zeichen A vorkommt, die Regel kann immer angewendet werden

$$S \rightarrow aAb \rightarrow aab$$

Regeln mit Kontext

Je nach **Kontext** kann eine Regel nicht unbedingt angewendet werden

$$S \rightarrow C \mid aC \mid bC$$
$$aC \rightarrow A$$
$$bC \rightarrow B$$
$$S \rightarrow aC \rightarrow A$$
$$S \rightarrow bC \rightarrow B$$
$$S \rightarrow C \rightarrow ?$$

CHOMSKY-NORMALFORM (CNF)

Eine CFG ist in **Chomsky-Normalform**, wenn S auf der rechten Seite nicht vorkommt und jede Regel von der Form $A \rightarrow BC$ oder $A \rightarrow a$ ist, zusätzlich ist die Regel $S \rightarrow \varepsilon$ erlaubt.

Umwandlung

1) Neue Startvariable $S_0 \rightarrow S$ (wenn nötig)

2) ε -Regeln: $A \xrightarrow{\varepsilon} AC \Rightarrow A$ kann weggelassen werden $\Rightarrow$

3) Unit-Rules: $A \xrightarrow{\varepsilon} B \Rightarrow AC$ $\Rightarrow$ aus A kann man wie aus B auch CD machen $\Rightarrow$ $A \xrightarrow{\varepsilon} CD$

4) Verkettungen: $A \rightarrow u_1 u_2 \dots u_n$ ersetzen durch $A \rightarrow u_1 A_1, A_1 \rightarrow u_2 A_2, \dots, A_{n-2} \rightarrow u_{n-1} u_n$ und falls u_i ein Terminalsymbol ist: $A_{i-1} \rightarrow u_i A_i, U_i \rightarrow u_i$.

Folgerungen

1) Ableitung eines Wortes $w \in L(G)$ ist immer in $2|w| - 1$ Regelanwendungen möglich. Beweis:

- $|w| - 1$ Regeln der Form $A \rightarrow BC$ um aus S ein Wort aus $|w|$ Variablen zu erzeugen
- $|w|$ Regeln der Form $A \rightarrow a$ um das Wort w zu erzeugen $\Rightarrow$ insgesamt $2|w| - 1$ Regelanwendungen

2) Deterministischer Parse-Algorithmus mit Laufzeit $O(|w|^3)$

$$S \rightarrow ASA \mid aB$$
$$A \rightarrow B \mid S$$
$$B \rightarrow b \mid \varepsilon$$

Beispiel 9:

1) Startvariable

$$S_0 \rightarrow S$$
$$S \rightarrow ASA \mid aB$$
$$A \rightarrow B \mid S$$
$$B \rightarrow b \mid \varepsilon$$

2) ε -Regel

$$S_0 \rightarrow S$$
$$S \rightarrow ASA \mid aB \mid a$$
$$A \rightarrow B \mid \varepsilon \mid S$$
$$B \rightarrow b$$
$$S_0 \rightarrow S$$
$$S \rightarrow ASA \mid SA \mid AS \mid aB \mid a$$
$$A \rightarrow B \mid S$$
$$B \rightarrow b$$

3) Unit-Regel

$$S_0 \rightarrow ASA \mid SA \mid AS \mid aB \mid a$$
$$S \rightarrow ASA \mid SA \mid AS \mid aB \mid a$$
$$A \rightarrow B \mid ASA \mid SA \mid AS \mid aB \mid a$$
$$B \rightarrow b$$
$$S_0 \rightarrow ASA \mid SA \mid AS \mid aB \mid a$$
$$S \rightarrow ASA \mid SA \mid AS \mid aB \mid a$$
$$A \rightarrow B \mid ASA \mid SA \mid AS \mid aB \mid a$$
$$B \rightarrow b$$

4) Komplexe Regeln

$$S_0 \rightarrow AA_1 \mid SA \mid AS \mid UB \mid a$$
$$S \rightarrow AA_1 \mid SA \mid AS \mid UB \mid a$$
$$A \rightarrow b \mid AA_1 \mid SA \mid AS \mid UB \mid a$$
$$A_1 \rightarrow SA$$
$$U \rightarrow a$$
$$B \rightarrow b$$

PARSING

COCKE-YOUNGER-KASAMI ALGORITHMUS (CYK)

Gegeben

1) Grammatik $G = (V, \Sigma, R, S)$

2) Variable $A \in V$

3) Wort $w \in \Sigma^*$

Frage

Ist w ableitbar? Formell geschrieben: $A \xRightarrow{*} w$

– Spezialfall $w = \varepsilon$: $A \xRightarrow{*} \varepsilon \Leftrightarrow A \rightarrow \varepsilon \in R$ (Epsilonregel, $A = S$)

– Spezialfall $|w| = 1$: $A \xRightarrow{*} w \Leftrightarrow A \rightarrow w \in R$ (Terminalsymbolregel)

– Fall $|w| > 1$:

$$A \xRightarrow{*} w \Rightarrow \exists \left\{ \begin{array}{l} A \rightarrow BC \in R \\ w = w_1 w_2 \\ w_i \in \Sigma^* \end{array} \right. \text{ mit } \left\{ \begin{array}{l} B \xRightarrow{*} w_1 \\ C \xRightarrow{*} w_2 \end{array} \right.$$

ABLEITUNGSDREIECK

Ideen

– Parse Tree aus $A \rightarrow BC$ und $T \rightarrow t$

– Variablen in Tabelle füllen

Prinzip

– Einem Teilwort entspricht ein Feld der Tabelle (rot hinterlegt)

– Das Feld wird mit den Variablen gefüllt, aus denen das Teilwort abgeleitet werden kann.

Beispiel

Parse des Wortes $()()()$

$$AB \mid CD \mid CU \mid SSB \rightarrow 1$$
$$SD$$
$$C \rightarrow [$$
$$D \rightarrow]$$

Definitionen

Die Felder (k, l_1) und $(k + l_1, l - l_1)$ heißen **korrespondierende Felder** im Ab-

leitungs-dreieck des Feldes (k, l) .

TODO:

rekursion vs iteration (buch 149)

BACKUS-NAUR-FORM (BNF)

Spezifikation der Regeln in maschinen-lesbarer Form:

– Variablen: **<variablen-name>**

– Einzelne Zeichen: A

– Zeichenketten: 'BEISPIEL'

– Regeln: **<variablen-name> ::= Ausdruck**

– Ausdrücke sind Folgen von Variablen, einzelnen Zeichen oder Zeichenketten, getrennt durch $|$

Beispiel 10: BNF für Expression-Term-Factor Grammatik

Ausgangsgrammatik

$$\text{expression} \rightarrow \text{expression} + \text{term} \mid \text{term}$$
$$\text{term} \rightarrow \text{term} * \text{factor} \mid \text{factor}$$
$$\text{factor} \rightarrow (\text{expression}) \mid N$$
$$N \rightarrow NZ \mid Z$$
$$Z \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$

Backus-Naur-Form

$$\langle \text{expression} \rangle ::= \langle \text{expression} \rangle + \langle \text{term} \rangle \mid \langle \text{term} \rangle$$
$$\langle \text{term} \rangle ::= \langle \text{term} \rangle * \langle \text{factor} \rangle \mid \langle \text{factor} \rangle$$
$$\langle \text{factor} \rangle ::= (\langle \text{expression} \rangle) \mid \langle \text{number} \rangle$$
$$\langle \text{number} \rangle ::= \langle \text{number} \rangle \langle \text{digit} \rangle \mid \langle \text{digit} \rangle$$
$$\langle \text{digit} \rangle ::= 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$

EXTENDED BACKUS-NAUR-FORM (EBNF)

Definition

– Literale in Anführungszeichen oder Apostroph

– = statt ::=

– Variablen ohne $<$ und $>$, dürfen Leerzeichen enthalten

– Komma für Verkettung

– Regel-Endzeichen ;

– Kommentare (* ... *)

– Optionale Wiederholung { ... }

– Option [...]

– Gruppierung (...)

– Ausnahme: -

Achtung!

– Keine Operatorpriorisierung

– Zweideutigkeit!

– Keine eindeutige Evaluation!

Beispiel 11: EBNF für Expression-Term-Factor Grammatik

$$\text{expression} = \text{expression}, "+", \text{term} \mid \text{term} \text{ ; } \varepsilon, \varepsilon \rightarrow \$$$
$$\text{term} = \text{term}, "*", \text{factor} \mid \text{factor} \text{ ; }$$
$$\text{factor} = "(", \text{expression}, ")" \mid \text{number} \text{ ; }$$
$$\text{number} = \text{digit}, \{ \text{digit} \} \text{ ; }$$
$$\text{digit} = "0" \mid "1" \mid "2" \mid "3" \mid "4" \mid "5" \mid "6" \mid "7" \mid "8" \mid "9" \text{ ; }$$

RECHTSREKURSION

Expression-Term-Factor-Grammatik verwendet Links-Rekursion $\Rightarrow$ korrekte Auswertung von links nach rechts. Parsertree wertet aber Ausdrücke von rechts nach links aus! Alternative Expression-Term-Factor definition mit Rechtsrekursion statt Linksrekursion:

$$\text{expression} \rightarrow \text{term expression}'$$
$$\text{expression}' \rightarrow + \text{term expression}' \mid \varepsilon$$
$$\text{term} \rightarrow \text{factor term}'$$
$$\text{term}' \rightarrow * \text{factor term}' \mid \varepsilon$$
$$\text{factor} \rightarrow (\text{expression}) \mid N$$
$$N \rightarrow NZ \mid Z$$
$$Z \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$

TODO:

grid $\rightarrow$ table

STACK

Unendlich grosser Speicher

– Immer nur das oberste Element sichtbar

– Beliebig tief

STACKAUTOMAT / PUSHDOWN AUTOMATON (PDA)

Kann Sprachen akzeptieren, die nicht regulär sind, ist aber nicht deterministisch. **Beachte:** $\Gamma \neq \Sigma$ möglich

Definition

Stackautomat $(Q, \Sigma, \Gamma, \delta, q_0, F)$

$P =$

Q : Zustände

Σ : Eingabe-Alphabet

Γ : Stack-Alphabet

δ : $Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \rightarrow P(Q \times \Gamma_\varepsilon)$

$q_0 \in Q$: Startzustand

$F \subset Q$: Akzeptierzustände

Übergänge

$a, b \rightarrow c$

a vom Input

b vom Stack entfernen (Bedingung)

c auf den Stack

Beispiel 12: Stackautomat für die Sprache $\{0^n 1^n \mid n \geq 0\}$

$$\begin{array}{c} q_0 \xrightarrow{\varepsilon, \varepsilon \rightarrow \$} q_1 \\ q_1 \xrightarrow{0, 0 \rightarrow \$} q_1 \\ q_1 \xrightarrow{1, \varepsilon \rightarrow \varepsilon} q_2 \\ q_2 \xrightarrow{\varepsilon, \$ \rightarrow \varepsilon} q_3 \end{array}$$

TODO:

Book 167-168, shift/reduce

TODO:

diagrams? (slides 8/12)

ABLEITUNG AUS CNF

Ist L eine kontextfreie Sprache, dann gibt es einen Stackautomaten P , der L akzeptiert, $L = L(P)$.

Grundgerüst

$$\begin{array}{c} S \xrightarrow{\varepsilon, \varepsilon \rightarrow S} R \\ R \xrightarrow{\varepsilon, A \rightarrow C} R \\ R \xrightarrow{\varepsilon, \varepsilon \rightarrow B} R \\ R \xrightarrow{\varepsilon, A \rightarrow a} R \\ R \xrightarrow{\varepsilon, S \rightarrow \varepsilon} R \\ R \xrightarrow{\varepsilon, \varepsilon \rightarrow R} R \\ R \xrightarrow{\varepsilon, a, a \rightarrow \varepsilon} R \end{array}$$

PDA $\rightarrow$ CFG

Variablen

Wörter beschreiben Pfade durch den Automaten: Variable A_{pq} = Wörter, die von p nach q führen mit leerem Stack

Regeln

Regeln beschreiben, wie sich Wege zerlegen lassen: $A_{pq} \rightarrow A_{pr} A_{rq}$ = Wege von p nach q können verstanden werden als Wege von p nach r und von dort nach q

Stackautomat standardisieren

TODO:

better diagrams, book 181

1) Nur ein Akzeptierzustand: Neuen Akzeptierzustand q_a und Übergänge von allen vorherigen Endzuständen zu q_a erstellen.

$\forall q \in F$: Start $\rightarrow q \rightarrow q_a$

2) Stack leeren: Mit einem Element $\$$ erweitern, sodass garantiert werden kann, dass der Stack am schluss leer ist.

$\varepsilon, \cdot \rightarrow \varepsilon$

$\varepsilon, \varepsilon \rightarrow \$$

$\varepsilon, \$ \rightarrow \varepsilon$

3) Jeder Übergang legt entweder ein Zeichen auf den Stack oder entfernt eines

$a, b \rightarrow c$

$a, b \rightarrow \varepsilon$

$a, \varepsilon \rightarrow \varepsilon$

$a, \varepsilon \rightarrow t$

TODO:

Book 182,183

Beispiel 13

TODO:

visualization

Grammatik ablesen

Ausgangspunkt: standardisierter PDA mit Startzustand q_0 und $F = \{q_a\}$.

1) Startvariable: A_{q_0, q_a}

2) Regeln:

$$\begin{array}{c} p \xrightarrow{A_{pp}} p \\ p \xrightarrow{A_{pq}} r \\ p \xrightarrow{A_{pr}} A \\ p \xrightarrow{A_{ps}} s \\ p \xrightarrow{A_{pq}} r \\ p \xrightarrow{A_{pr}} A \\ p \xrightarrow{A_{ps}} s \\ p \xrightarrow{A_{pq}} r \\ p \xrightarrow{A_{pr}} A \\ p \xrightarrow{A_{ps}} s \end{array}$$

Beispiel 14

TODO:

NICHT KONTEXTFREIE SPRACHEN

PUMPING LEMMA

Pumping Lemma für CFL

Ist L eine CFL, dann gibt es eine Zahl N , die die Pumping Length, derart, dass jedes Wort $w \in L$ mit $|w| \geq N$ zerlegt werden kann in fünf Teile $w = uvxyz$ derart, dass

1) $|vxy| > 0$

2) $|vxy| \leq N$

3) $uv^k xy^k z \in L \forall k \in \mathbb{N}$

REDUKTION

Reduktionsabbildung

Berechenbare Abbildung $f : \Sigma^* \rightarrow \Sigma^*$ so, dass

$$w \in A \Leftrightarrow f(w) \in B$$

Notation: $f : A \leq B$ heisst « A leichter entscheidbar als B »

Entscheidbarkeit

Falls B entscheidbar, dann $f : A \leq B \Rightarrow A$ entscheidbar

Beweis

Ist H ein Entscheider für B , dann ist $H \circ f$ ein Entscheider für A .

Folgerung

A nicht entscheidbar, $A \leq B \Rightarrow B$ nicht entscheidbar

VERGLEICH DER ENTSCHEIDBARKEIT VON SPRACHEN

Beispiel 18

TODO:

SATZ VON RICE

LÖSUNGSMETHODEN FÜR ENTSCHEIDUNGSPROBLEME

TODO:

book 273..

KOMPLEXITÄT

Ineffizient: Entscheidbare Probleme mit exorbitant langer Laufzeit sind praktisch «nicht lösbar»

Problemgrösse: Laufzeit hängt von der Problemgrösse n ab

Worst case Laufzeit: Die worst case Laufzeit $t(n)$, die die Maschine zur Verarbeitung von Inputs der Länge n braucht, ist $t(n) = \max\{t(w) \mid |w| \leq n\}$

HARDWAREEINFLUSS

Eine Sprache L gehört zur Klasse NP , wenn L von einer nichtdeterministischen TM in **polynomieller Zeit entschieden** werden kann, $t(n) = O(n^k)$

TODO:

slides 3,4

POLYNOMIELLE UND EXPONENTIELLE LAUFZEIT

TODO:

slides 5

Polynomielle Probleme	Exponentielle Probleme
– Gauss-Algorithmus $O(n^3)$ – ...	– Faktorisierung von grossen Zahlen (RSA) – ...
Skalierbarkeit	Sicherheit

HARDWAREUNABHÄNGIGE LAUFZEIT-KOMPLEXITÄT

Ein Algorithmus hat **polynomielle Laufzeit**, wenn seine Laufzeit durch ein Polynom beschränkt ist: $t(n) = O(n^k)$.

NP

Eine Sprache gehört zur Klasse **NP**, wenn sie von einem nichtdeterministischen Entscheider in polynomieller Zeit entschieden werden kann

POLYNOMIELLE VERIFIZIERER

Ein **polynomieller Verifizierer** für die Sprache L ist eine TM V , so dass es für jedes $w \in \Sigma^*$ ein Wort c (das Lösungszertifikat) gibt, für das gilt

$$w \in L \Leftrightarrow V \text{ akzeptiert } \langle w, c \rangle$$

Die Laufzeit von V ist polynomiell in $|w|$.

Eine Sprache ist genau dann in **NP**, wenn sie in polynomieller Zeit **verifiziert** werden kann.

TODO:
slides 14

POLYNOMIELLE REDUKTION

POLYNOMIELLER LAUFZEIT-VERGLEICH

Seien A und B entscheidbare Sprachen. Eine berechenbare Abbildung

$f : \Sigma^* \rightarrow \Sigma^* : w \mapsto f(w)$

mit

- $w \in A \Leftrightarrow f(w) \in B$ (Reduktion)
- $f(w)$ ist berechenbar in polynomieller Zeit in $|w|$

Entscheidbare **polynomielle Reduktion** $A \leq_p B$. Lies: A ist polynomiell leichter entscheidbar als B .

Polynomiell äquivalent: Zwei Sprachen A und B heissen **polynomiell äquivalent**, geschrieben $A \equiv_p B$, wenn $A \leq_p B$ und $B \leq_p A$.

SATZ

Sei $A \leq_p B$. Dann gilt

$$B \in P \Rightarrow A \in P, \quad A \notin P \Rightarrow B \notin P$$
$$B \in NP \Rightarrow A \in NP, \quad A \notin NP \Rightarrow B \notin NP$$

TODO:

slides 16,17

REDUKTION VON HAMPATH

TODO:

POLYNOMIELLE AUSFÜLLRÄTSEL

Definition

Variablen

Ein polynomieller **Ausfüllrät** φ $\Leftrightarrow$ Feld (i, j) der Tabelle enthält Zeichen c

sel ist eine $n \times m$ -Tabelle, in die Zeichen eines Alphabets Σ eingefüllt werden müssen, so dass als logische Formel ausdrückbare Regeln eingehalten werden, die in polynomieller Zeit ausgewertet werden können.

Genau ein Zeichen im Feld (i, j)

$$\varphi_{ij} = \bigvee_{c \in \Sigma} (\underbrace{x_{ijc} \wedge \bigvee_{d \neq c} x_{ijd}}_{c \text{ und kein anderes Zeichen in } (i,j)}$$

Regeln

Satz

Jedes polynomielle Ausfüllrätzel A lässt sich polynomiell auf **SAT** reduzieren.

Beispiel

$n^2 \times n^2$ – Sudoku